



SHEIKENS MAILBUS

Kravspekifikation

Af
Niklas Kim Jensen
Og
Sammy Damiri

H5 - Datatekniker med Speciale i
Programmering
Technical Education Copenhagen,
Ballerup

Vejleder: Henrik Vincents Poulsen

Indholdsfortegnelse

1. Indledning.....	2
1.1 Formål	2
1.2 Referencer i form af bilag	2
1.3 Læsevejledning.....	2
2. Generel beskrivelse	3
2.1 Systembeskrivelse	3
2.2 Systemets funktioner	3
2.3 Systemets begrænsninger	5
2.4 Systemets fremtid	5
2.5 Brugerprofil	5
3. Specifikke krav	6
3.1 Definitioner.....	6
3.2 Funktionelle krav.....	6
3.3 Ikke-funktionelle krav	7
4. Accepttest	8
5. Dokument Historik	10

1. Indledning

1.1 Formål

Denne kravspecifikation beskriver kravene til et cloudbaseret mailsystem udviklet som en del af vores H5 projekt.

Formålet med systemet er at demonstrere den infrastruktur der kræves for at modtage, behandle og vise mails på eget domæne. Systemet skal kunne tilgås sikkert via en webbrowser og et REST API og skal kunne deployes automatisk.

Kravspecifikationen er udarbejdet af projektgruppen og danner grundlag for udviklingen og den afsluttende prøve.

1.2 Referencer i form af bilag

1. Bilag_Problemformulering.pdf
2. Bilag_UseCases.pdf
3. Bilag_UserStories.pdf

1.3 Læsevejledning

Kapitel 2 giver en general beskrivelse af systemet, herunder systembeskrivelse, aktører, begrænsninger og brugerprofil.

Kapitel 3 indeholder de specifikke krav opdelt i funktionelle og ikke-funktionelle krav.

Kapitel 4 beskriver accepttestene der bruges til at verificere at kravene er opfyldt.

Kapitel 5 er dokumentets versions historik.

Bilag_Problemformulering.pdf indeholder problemformulering.

Bilag_UseCases.pdf indeholder use cases.

Bilag_UserStories.pdf indeholder user stories.

2. Generel beskrivelse

2.1 Systembeskrivelse

Som del af vores H5 projekt har vi bygget en prototype på et mailsystem til en organisation med et maildomæne.

Når en mail kommer udefra, tager SendGrid imod den og videresender den til systemet. Den lægges i en kø, behandles og ender i den rette bruges indbakke. Brugeren logger ind via en browser og kan læse sine mails derfra. Udviklere kan tilgå maildata direkte gennem et API, fx fra Postman.

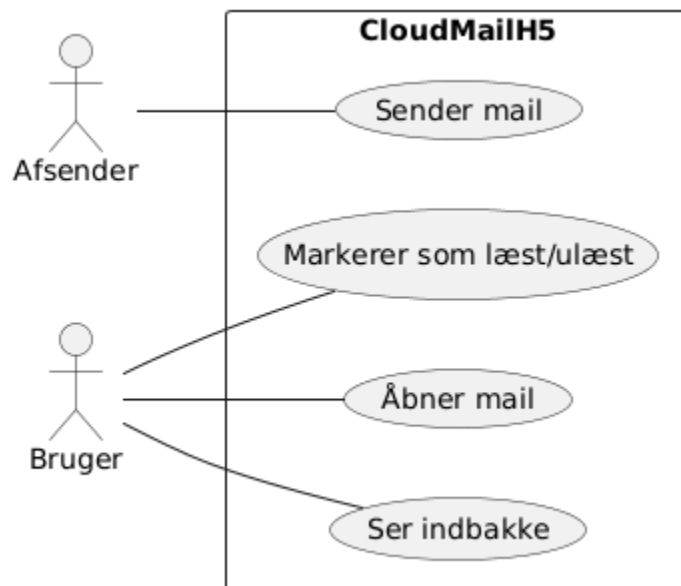
Systemet kører på Azure. Azure Service Bus håndterer køen. Azure SQL gemmer data. Azure App Service hoster applikationen.

Login klares med Microsoft Identity/Entra ID for både brugere og udviklere. Backendten er skrevet i ASP.NET Core, fronten i Blazor.

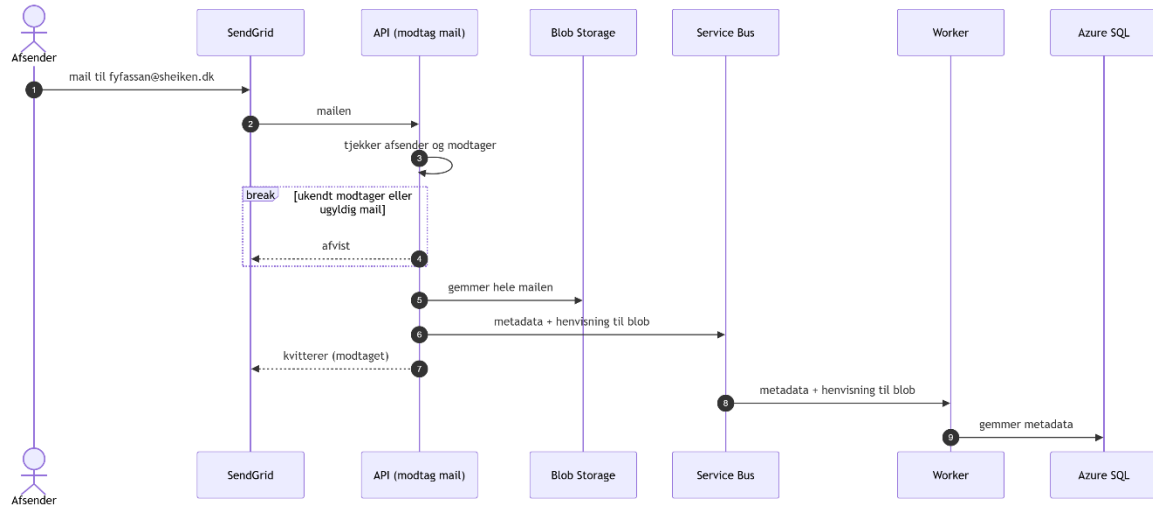
Til lokal udvikling bruger vi .NET Aspire til bl.a service discovery og for at få et bedre overblik under udviklingen.

2.2 Systemets funktioner

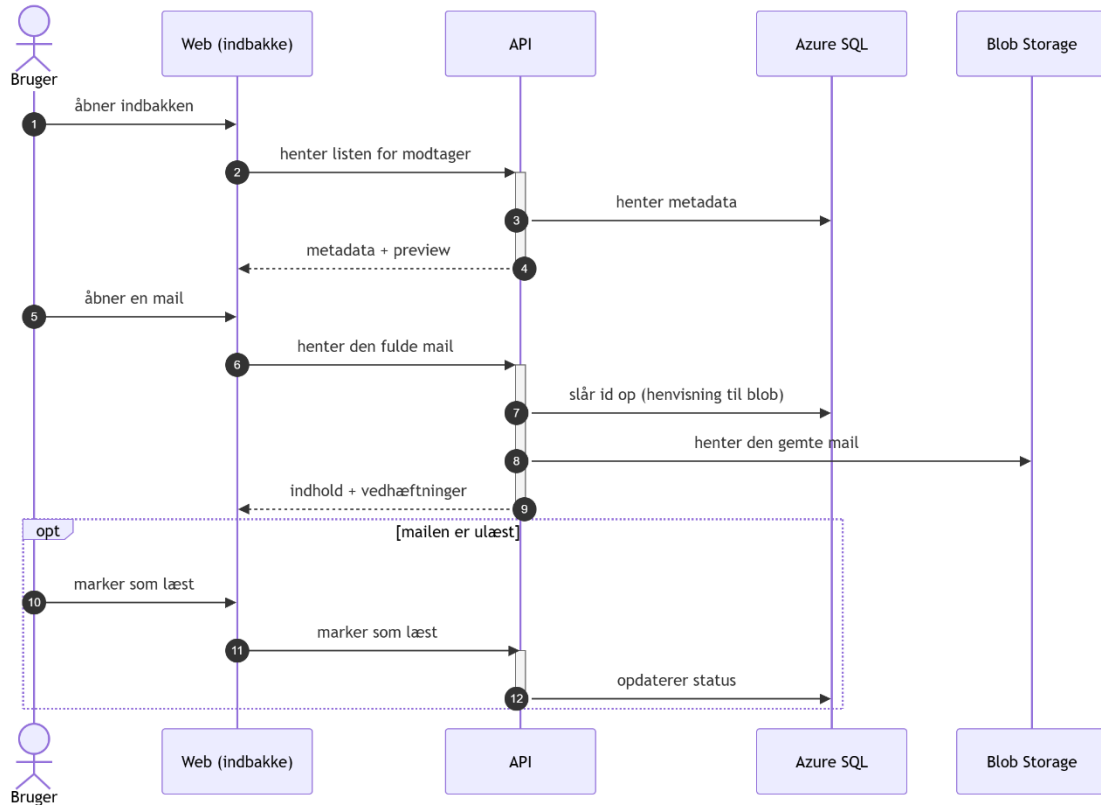
Systemets funktioner er beskrevet ved hjælp af use case teknikken og er dokumenteret i Bilag_UseCases.pdf.



Mail sendt til vores system udefra.



Bruger interaktion med vores system.



2.3 Systemets begrænsninger

Systemet er en prototype og følgende er bevidst holdt uden for scope.

Systemet understøtter kun modtagelse og visning af indgående mails. Afsendelse er ikke implementeret, selvom det er en naturlig forventning til et mailsystem, ligger det udenfor hvad vi kommer til at demonstrere til denne minisvend.

Klienten tilgås via webbrowser som kun kan tilgås online.

Systemet understøtter 1 organisation med 1 maildomæne. Som er tilstrækkeligt til at demonstrere de centrale funktioner.

Spamfilter er ikke i scope. Mails valideres på domæne, ikke indhold.

Kalender integration er også ude af scope.

2.4 Systemets fremtid

Systemet er bygget med mulighed for udvidelse. Følgende funktioner vil være et naturligt næste skridt.

Hente vedhæftede filer. Disse bliver allerede gemt i Blob Storage.

Brugeren skal have mulighed for at besvare en indgående mail.

Brugeren skal have mulighed for at lave en ny mail og sende den.

Simpel søgning på afsender, emne og indhold.

Infrastrukturen kan automatisk opsættes via Bicep skabeloner fremfor manuelt i Azure.

2.5 Brugerprofil

Systemet har 3 brugergrupper.

Brugeren er en medarbejder i en organisation med en mailadresse på domænet. Brugeren forventes at have grundlæggende erfaring med webbrowser og mailklienter, men ingen teknisk viden om systemets infrastruktur.

Udvikleren er en teknisk person der tilgår maildata direkte via API'et fx fra Postman.

Udvikleren forventes at have erfaring med HTTP kald og JWT authentication.

DevOps er ansvarlig for opsætning og konfiguration af Azure infrastrukturen, registrering af domænet og oprettelse af brugere. DevOps forventes at have erfaring med Azure og CI/CD pipelines.

3. Specifikke krav

3.1 Definitioner

JWT (JSON Web Token): En token bliver givet ved login og beviser hvem brugeren er og hvad de har adgang til.

MoSCoW: En prioritering der deler krav op i fire kategorier. Must have, Should have, Could have og Won't have. Det bruges til at afklare hvad der er vigtigt nu og hvad der kan vente.

3.2 Funktionelle krav

Funktionelle krav beskriver hvad systemet skal kunne. Hvert krav er nummereret med KF-præfiks og kan spores til en user story i Bilag_UserStories.pdf.

Prioriteringen følger MoSCoW.

Must Have:

KF1: Mails fra domæner der ikke er registreret, skal afvises og logges

KF2: DevOps skal kunne registrere domænet og oprette brugere

KF3: En bruger skal kunne logge ind og tilgå sin indbakke

KF4: Indbakken skal vise afsender, emne og tidspunkt for hver mail, sorteret med nyeste øverst

KF5: En bruger skal kunne åbne en mail og læse indholdet

KF6: En mail skal markeres som læst første gang brugeren åbner den

KF7: En udvikler skal kunne logge ind via Microsoft Identity/Entra ID og modtage en JWT

KF8: En udvikler skal kunne hente mailliste og mailindhold via API'et med en gyldig JWT

Should Have:

KF9: Nye mails skal vises automatisk i indbakken uden at siden genindlæses

KF10: En bruger skal kunne slette en mail

KF11: Ulæste mails skal markeres visuelt i indbakken

KF12: Kode skal automatisk bygges og deployes til Azure når et pull request merges til main

Could Have:

KF13: En bruger skal kunne gemme vedhæftninger fra en mail

KF14: En bruger skal kunne søge i sine mails på afsender, emne eller indhold

KF15: API'et skal have tilgængelig dokumentation via Swagger

KF16: Infrastrukturen skal kunne provisioneres fra Bicep-skabeloner

3.3 Ikke-funktionelle krav

Ikke-funktionelle krav beskriver hvordan systemet opfører sig, ikke hvad det gør.

Tilgængelighed:

KNF1: Indkommende mails skal gemmes i kø og leveres til indbakken, selv om backend midlertidigt er nede

Sikkerhed:

KNF2: En bruger må ikke kunne tilgå en anden brugers mails

KNF3: Al trafik mellem klient og server skal gå over HTTPS

KNF4: Login skal ske via Microsoft Identity/Entra ID for både brugere og udviklere

Observability:

KNF5: Fejl og traces skal logges via Application Insights og OpenTelemetry

Drift og teknologi:

KNF6: Systemet skal køre på Azure

KNF7: Backend skrives i ASP.NET Core og frontend i Blazor. .NET Aspire bruges til service discovery og observability lokalt

4. Accepttest

Accepttestene viser konkret hvad der skal til for at erklære et krav opfyldt.

KF1: Mails fra uregistrerede domæner afvises. Det er kun SendGrid der accepters

Test: En mail fra et uregistreret domæne/mailudbyder postes til Api'et

Forventet resultat: Mailen kasseres og logges. Ingen mail gemmes eller vises i indbakken.

KF3: Bruger logger ind

Test: Log ind med korrekte oplysninger

Forventet resultat: Brugeren sendes til indbakken

Test: Log ind med forkert password

Forventet resultat: Fejlbesked vises, ingen adgang gives

KF4: Indbakke viser mails sorteret med nyeste øverst

Test: Åbn indbakken med flere mails

Forventet resultat: Afsender, emne og tidspunkt vises for hver mail

Test: Modtag to mails på forskellige tidspunkter

Forventet resultat: Den nyeste vises øverst

KF5: Bruger læser mail

Test: Klik på en mail i indbakken

Forventet resultat: Mailens indhold vises

KF6: Mail markeres som læst

Test: Åbn en ulæst mail

Forventet resultat: Mailen markeres som læst

KF7: Udvikler logger ind

Test: Login med Microsoft Identity/Entra Id og få et access token via Postman OAuth

Forventet resultat: JWT returneres

Test: Brug JWT i et API kald

Forventet resultat: API svare med http status kode 200.

Test: Brug en udløbet JWT

Forventet resultat: API'et afviser med 401

KF8: Udvikler henter mailliste og mailindhold via API

Test: Kald GET /emails med gyldig JWT

Forventet resultat: Mailliste returneres

Test: Kald GET /emails/{id} med gyldig JWT

Forventet resultat: Mailindhold returneres

Test: Kald GET /emails uden JWT

Forventet resultat: API'et afviser med 401

KNF2: Bruger kan ikke tilgå en anden brugers mails

Test: Log ind som Bruger A og forsøg at hente Bruger B's mails via API

Forventet resultat: API'et afviser med 403 eller returnerer tomt resultat

KNF4: Login sker via Entra ID

Test: Log ind som bruger via hjemmesiden

Forventet resultat: Login sker via Entra ID login-siden

Test: Log ind som udvikler via Postman OAuth

Forventet resultat: Token udleveres af Entra ID

5. Dokument Historik

Version 1.0.0 – 15-06-26